Retro Arcade Puzzle · Hex Grid Strategy · Two Game Modes · 10 Levels

HTML5 | CSS3 | JAVASCRIPT | DEPENDENCIES ZERO | LICENSE MIT

A neon-drenched hexagonal puzzle game — trap the rogue dot before it breaks free.

Play Now · Explore the Code · Contribute

Table of Contents

1. What Is This?
2. How to Run Locally
3. How to Play
4. Codebase Deep Dive
   - Project Structure
   - The Game State
   - The Hex Grid Engine
   - BFS Pathfinding — The Dot's Brain
   - Two Game Modes
   - The Animated Background Canvas
   - The Retro UI Layer
5. Tech Stack
6. Level Configuration
7. Future Improvements
8. Author

What Is This?

Catch The Dot is a browser-based, zero-dependency retro arcade puzzle. The objective is disarmingly simple: place orange blockers on a hexagonal grid to completely surround a rogue blue dot before it escapes to the edge.

What makes it deceptively hard is the opponent. The dot is not random — it uses **BFS (Breadth-First Search)** to always find the shortest possible escape route and advances along it every time you place a blocker. Trap it efficiently and you're an arcade champion. Hesitate, and it slips through the gap you forgot to close.

The game ships with:

- **10 escalating Normal Mode levels** — grids grow from 7×7 to 12×12, pre-seeded blockers thin out, and the dot gets faster
- **1 infinite Timed Mode** — 60 seconds, auto-advancing dot pulses, stage-cleared counters, close-block bonuses (+1.2s), and escape penalties (−5s)
- **A living neon retro aesthetic** — CRT scanline flicker, neon drop-shadows, animated starfield, chromatic aberration headings, and a full synthwave color palette — all in pure CSS
- **A per-session personal best tracker** — fewest moves per level in Normal Mode and most stages cleared per run in Timed Mode

---

How to Run Locally

No build step. No npm. No config. Just a browser.

Option 1 — Open directly

```
# Clone the repository
git clone https://github.com/danish1323/catch-the-dot.git
cd catch-the-dot

# macOS
open index.html

# Linux
xdg-open index.html
```

```
# Windows (PowerShell)
start index.html
```

Option 2 — Local dev server *(recommended — avoids browser restrictions on local file loading)*

```
# Python 3
python3 -m http.server 8080

# Node.js (no install needed)
npx serve .

# VS Code
# Install the "Live Server" extension  right-click index.html  "Open with Live Serve
```

Then navigate to `http://localhost:8080` in any modern browser.

> **Compatibility:** Chrome 90+, Firefox 88+, Safari 14+, Edge 90+ **Mobile:** Fully touch-responsive. Works on iOS Safari and Android Chrome. **Offline:** No CDN calls for game logic. Only Google Fonts requires internet.

How to Play

Normal Mode

1. Pick a level (1–10) and click **Play** **Start Level**
2. The board appears with a **blue dot** at the center, some cells already blocked
3. **Click any empty gray cell** to drop an orange blocker
4. After your click, the dot uses BFS to find the shortest path to any edge and **moves one step along it**
5. Keep placing blockers to cut off every escape route
6. **Win** dot has zero path to any edge (fully surrounded)
7. **Lose** dot reaches any edge cell before you trap it

Timed Mode

1. Select **Timed Mode** and click **Play** → **Start Level**
2. You get **60 seconds** and an endless sequence of stages
3. The dot moves **automatically on a 1.1-second pulse**, regardless of your clicks
4. Place a blocker **adjacent to the dot** → earn **+1.2s bonus time**
5. Dot escapes → **−5.0s penalty**, next stage auto-loads
6. Trap the dot → stage cleared, next stage auto-loads. Clear as many as possible before time runs out

Tips

- Prioritize closing the **nearest edge openings** first — the dot always takes the shortest available path
- In Timed Mode, **close-risk blocks adjacent to the dot** are doubly valuable: they buy you time and cut escape routes simultaneously
- Watch the **HUD hint line** — each level and stage surfaces a unique strategic tip

---

Codebase Deep Dive

Project Structure

```
catch-the-dot/
    index.html    # Markup: 3 screen sections (start/instructions/game), SVG boar
    styles.css    # Full neon retro aesthetic: layout, animations, CRT effects, respor
    script.js     # All game logic: BFS pathfinding, hex grid, state machine, canvas s
```

The entire game lives in these three files with **zero external runtime dependencies** — not even a utility library. Every pixel, every animation, and every algorithm is hand-rolled.

The Game State

Every aspect of the game lives in a single `state` object, making it easy to reason about, serialize, and debug:

```
// script.js
let state = {
  selectedLevelIndex: 0,
  selectedMode: "normal",
  levelIndex: 0,
  mode: "normal",
  board: [],          // 2D array of CELL.EMPTY | CELL.BLOCKED | CELL.DOT
  size: 0,            // Grid dimension (7–12 depending on level)
  dot: { row: 0, col: 0 },
  moves: 0,
  isGameOver: false,
  bestByKey: {},      // Personal best scores, keyed by "mode-levelIndex"
  timeLeftMs: 0,      // Timed mode countdown in milliseconds
  pulseIntervalId: null,
  timerIntervalId: null,
  stageClearedCount: 0,
  stageNumber: 1,
};
```

Board cells use named constants instead of bare magic numbers, making every condition self-documenting:

```
// script.js
const CELL = { EMPTY: 0, BLOCKED: 1, DOT: 2 };
```

The Hex Grid Engine

The board isn't a standard square grid — it's a **hexagonal grid rendered entirely in SVG**, with alternating row offsets that create the staggered honeycomb layout. The rendering function recomputes the SVG `viewBox` dynamically based on the current grid size, so every level looks perfectly proportioned with no hardcoded pixel values:

```
// script.js
function renderBoard() {
  boardEl.innerHTML = "";
```

```
  const radius = 17;
  const gap = 5;
  const xStep = radius * 2 + gap;
  const yStep = Math.sqrt(3) * radius + gap;  // vertical step for hex geometry
  const margin = radius + 8;

  const totalWidth  = margin * 2 + (state.size - 1) * xStep + radius;
  const totalHeight = margin * 2 + (state.size - 1) * yStep + radius;
  boardEl.setAttribute("viewBox", `0 0 ${totalWidth} ${totalHeight}`);

  for (let row = 0; row < state.size; row++) {
    for (let col = 0; col < state.size; col++) {
      // Odd rows are offset right by half a step — this creates the hex stagger
      const cx = margin + col * xStep + (row % 2 === 1 ? xStep / 2 : 0);
      const cy = margin + row * yStep;

      const circle = document.createElementNS(SVG_NS, "circle");
      circle.setAttribute("cx", String(cx));
      circle.setAttribute("cy", String(cy));
      circle.setAttribute("r", String(radius));
      circle.classList.add("cell");
      // ...
    }
  }
}
```

Hex neighbor calculation is the backbone of both movement and pathfinding. Even and odd rows have different neighbor offsets due to the column stagger:

```
// script.js
function getHexNeighbors(row, col) {
  const evenRowOffsets = [
    { dr: -1, dc: -1 }, { dr: -1, dc: 0 },
    { dr:  0, dc: -1 }, { dr:  0, dc: 1 },
    { dr:  1, dc: -1 }, { dr:  1, dc: 0 },
  ];

  const oddRowOffsets = [
    { dr: -1, dc: 0 }, { dr: -1, dc: 1 },
```

```
  { dr:  0, dc: -1 }, { dr:  0, dc: 1 },
  { dr:  1, dc: 0 }, { dr:  1, dc: 1 },
 ];

 const offsets = row % 2 === 0 ? evenRowOffsets : oddRowOffsets;
 return offsets.map((o) => ({ row: row + o.dr, col: col + o.dc }));
}
```

BFS Pathfinding — The Dot's Brain

This is the heart of the game. The dot is not random — it uses **Breadth-First Search** to find the shortest unblocked path to any edge cell. After you place a blocker, the dot calls `findPathToNearestEdge()`, which explores the hex graph outward from the dot's position. The moment BFS reaches an edge cell, it reconstructs the shortest path via a parent map and the dot moves to the very first step of that path.

```
// script.js
function findPathToNearestEdge(startRow, startCol) {
 const queue = [{ row: startRow, col: startCol }];
 const seen = new Set([key(startRow, startCol)]);
 const parent = new Map();

 while (queue.length > 0) {
  const current = queue.shift();

  // As soon as we hit any edge, reconstruct and return the path
  if (isEdge(current.row, current.col)) {
   return rebuildPath(current, parent);
  }

  for (const next of getHexNeighbors(current.row, current.col)) {
   if (!isInside(next.row, next.col)) continue;
   if (state.board[next.row][next.col] === CELL.BLOCKED) continue; // blockers are

   const nextKey = key(next.row, next.col);
   if (seen.has(nextKey)) continue;
   seen.add(nextKey);
   parent.set(nextKey, current);
```

```
    queue.push(next);
  }
 }

 return null; // No path found = dot is completely surrounded = player wins
}
```

Win detection is elegantly reduced to a single line by reusing the same BFS:

```
// script.js
function isDotTrapped() {
 return !findPathToNearestEdge(state.dot.row, state.dot.col);
}
```

When `findPathToNearestEdge` returns `null`, the dot has no reachable edge — the player wins. No separate win-detection logic needed.

Two Game Modes

Both modes share the same board, hex engine, and BFS logic, but differ fundamentally in *when* the dot moves and *what* failure means.

Normal Mode — turn-based, fully player-driven. The dot moves exactly once per blocker placed:

```
// script.js — onCellClick()
if (state.mode === "normal") {
 moveDot(); // dot moves ONLY after the player acts

 if (isEdge(state.dot.row, state.dot.col)) {
  onLevelLost("The dot escaped. Restart and try a tighter block.");
  renderBoard();
  return;
 }

 if (isDotTrapped()) {
  onLevelWon();
  return;
```

```
  }

  setMessage("Nice move. Keep closing the nearest exits.", "");
}
```

Timed Mode — real-time, clock-driven. The dot moves autonomously on a 1.1-second pulse regardless of what the player does. Escape costs 5 seconds; a close block near the dot rewards 1.2 seconds, incentivizing aggressive high-risk play:

```
// script.js — startTimedSession()
state.pulseIntervalId = window.setInterval(() => {
  if (state.timeLeftMs <= 0) return;

  moveDot();    // dot moves on its own — no player action required
  renderBoard();

  if (isEdge(state.dot.row, state.dot.col)) {
    onLevelLost("Time mode: the dot pulsed to the edge and escaped.");
    return;
  }

  if (isDotTrapped()) {
    onLevelWon(); // auto-advance to next stage immediately
    return;
  }
}, 1100 /* ms */);
```

The close-block bonus — one of the game's cleverest design choices — is checked against `getHexNeighbors` to determine adjacency:

```
// script.js — onCellClick()
if (state.mode === "timed" && isNearDot(row, col)) {
  state.timeLeftMs += 1200;
  setMessage("Close block bonus: +1.2s", "");
}

function isNearDot(row, col) {
  return getHexNeighbors(state.dot.row, state.dot.col).some(
```

```
    (neighbor) => neighbor.row === row && neighbor.col === col
  );
}
```

The Animated Background Canvas

The background isn't a static CSS image — it's a live `<canvas>` animation running at `requestAnimationFrame` speed, layered behind the CSS gradient. 70 stars drift downward at randomized speeds and radii, respawning at the top when they leave the bottom of the screen. Horizontal scanline bands paint over the top to reinforce the CRT aesthetic:

```
// script.js — initBackgroundCanvas()
function drawBackground() {
  const width = window.innerWidth;
  const height = window.innerHeight;

  ctx.clearRect(0, 0, width, height);

  // Drift stars downward; respawn at top when they exit the screen
  for (const star of stars) {
    star.y += star.speed;
    if (star.y > height + 4) {
      star.y = -4;
      star.x = Math.random() * width;
    }

    ctx.beginPath();
    ctx.fillStyle = "rgba(255, 244, 195, 0.68)";
    ctx.arc(star.x, star.y, star.r, 0, Math.PI * 2);
    ctx.fill();
  }

  // Horizontal CRT scanline bands — a signature of the aesthetic
  for (let y = 0; y < height; y += 18) {
    ctx.fillStyle = "rgba(41, 22, 109, 0.08)";
    ctx.fillRect(0, y, width, 7);
  }
```

```
  window.requestAnimationFrame(drawBackground);
}
```

The canvas is fully DPI-aware (Retina displays render crisp) and reacts to window resizes:

```
// script.js
function resizeCanvas() {
  const dpr = window.devicePixelRatio || 1;
  bgCanvas.width  = Math.floor(window.innerWidth  * dpr);
  bgCanvas.height = Math.floor(window.innerHeight * dpr);
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
}

window.addEventListener("resize", resizeCanvas);
```

The Retro UI Layer

The entire neon arcade aesthetic is pure CSS — no image assets, no sprite sheets, no external libraries. Three key techniques drive the visual identity:

CRT scanline flicker — a `::before` pseudo-element with a `steps()` animation simulates the phosphor flicker of vintage arcade monitors:

```
/* styles.css */
body::before {
  background:
    repeating-linear-gradient(
      180deg,
      rgba(0, 0, 0, 0) 0 2px,
      rgba(26, 255, 214, 0.09) 2px 4px
    ),
    linear-gradient(90deg, rgba(255, 0, 128, 0.06), rgba(0, 255, 255, 0.03));
  mix-blend-mode: screen;
  animation: crt-flicker 3.2s steps(12) infinite;
}
```

```
@keyframes crt-flicker {
  0%, 100% { opacity: 0.36; }
  50%      { opacity: 0.48; }
}
```

Neon cell glow — every SVG element uses CSS `drop-shadow` filters for the signature neon bloom, with the dot and blockers having distinct color signatures:

```
/* styles.css */
.board-svg {
  filter: drop-shadow(0 0 11px rgba(0, 255, 255, 0.22));
}

.cell.dot {
  fill: #ff376f;
  stroke: #ffc2d3;
  filter: drop-shadow(0 0 10px rgba(255, 55, 111, 0.78));  /* hot pink */
}

.cell.blocker {
  fill: var(--accent-hot);
  stroke: #ffe29a;
  filter: drop-shadow(0 0 7px rgba(255, 164, 56, 0.62));   /* orange */
}
```

Chromatic aberration on headings — the title glow uses offset `text-shadow` in red and cyan to mimic RGB channel misalignment on old CRT displays:

```
/* styles.css */
h1, h2 {
  color: #fff38b;
  text-shadow:
    -2px 0 #ff2b78,                    /* red channel shift left  */
     2px 0 #2affff,                    /* cyan channel shift right */
     0 0 12px rgba(255, 228, 106, 0.6);     /* warm glow            */
}
```

Design tokens — the full palette lives in CSS custom properties for clean, consistent theming:

```
/* styles.css */
:root {
  --bg-top:      #190024;
  --bg-bottom:   #04020d;
  --ink:         #d7fff9;
  --accent-main: #ff2b78;
  --accent-hot:  #ff9a1f;
  --accent-cool: #2affff;
  --accent-lime: #95ff4b;
  --tile:        #1f2e4d;
  --tile-stroke: #6ad2ff;
  --danger:      #ff5b74;
  --ok:          #75ff9a;
}
```

Tech Stack

Technology	What it powers
HTML5 `<svg>`	The entire game board — all cells are SVG `<circle>` elements created dynamically via `createElementNS`
HTML5 `<canvas>`	The animated starfield and CRT scanline background layer
CSS Custom Properties	Global design tokens for every color, shadow, and radius in the UI
CSS Grid	HUD layout, level picker, responsive column collapsing across breakpoints
CSS Flexbox	Button rows, mode picker, navigation
CSS `clamp()`	Fluid typography — font sizes scale continuously between viewport extremes
CSS Animations	CRT flicker via `steps()`, marquee blink, hover glow transitions
`backdrop-filter: blur`	Frosted glass effect on cards and panels
`mix-blend-mode: screen`	CRT overlay compositing on top of background layers

`drop-shadow()` filter	Per-element neon bloom on all SVG cells
Vanilla JS BFS	Dot pathfinding engine and win-condition detection
`setInterval`	Timed mode dot pulse (1.1s) and countdown timer (100ms tick)
`requestAnimationFrame`	Starfield canvas animation loop
`devicePixelRatio`	Retina-aware, pixel-perfect canvas rendering
`createElementNS`	Dynamic SVG element generation for the game board
Google Fonts	`Press Start 2P` — the pixel font that defines the entire visual identity

---

Level Configuration

All 10 levels are defined in a single declarative array. Each property directly controls a specific mechanical lever of difficulty:

```
// script.js
const LEVELS = [
  { size: 7,  randomBlockers: 14, time: 55, pulse: 1600 }, // Level 1  — generous help,
  { size: 7,  randomBlockers: 13, time: 53, pulse: 1560 }, // Level 2
  { size: 8,  randomBlockers: 15, time: 50, pulse: 1500 }, // Level 3  — grid grows
  { size: 8,  randomBlockers: 13, time: 48, pulse: 1450 }, // Level 4  — fewer helpers
  { size: 9,  randomBlockers: 14, time: 44, pulse: 1380 }, // Level 5
  { size: 9,  randomBlockers: 15, time: 42, pulse: 1320 }, // Level 6
  { size: 10, randomBlockers: 16, time: 39, pulse: 1260 }, // Level 7  — large board
  { size: 10, randomBlockers: 17, time: 36, pulse: 1200 }, // Level 8
  { size: 11, randomBlockers: 19, time: 33, pulse: 1130 }, // Level 9
  { size: 12, randomBlockers: 21, time: 30, pulse: 1060 }, // Level 10 — maximum pre
];
```

Property	Effect
`size`	Grid dimension (N×N). Larger grids have more cells and exponentially more potential escape routes
`randomBlockers`	Pre-placed blockers at level start. Fewer helpers = harder puzzle requiring more player planning
`time`	Timed Mode per-stage time budget in seconds before the stage fails

`pulse`	Timed Mode dot movement interval in milliseconds. Smaller = faster autonomous dot movement

Timed Mode stages escalate infinitely beyond Level 10 using a dynamic scaling function that blends base difficulty with stage depth:

```
// script.js — setupTimedStage()
function setupTimedStage(stageNumber) {
  const difficulty = Math.min(LEVELS.length - 1, Math.floor((stageNumber - 1) / 2));
  const base = LEVELS[difficulty];
  // Grid grows every 4 stages, capped at 12×12
  const size = Math.min(12, base.size + Math.floor((stageNumber - 1) / 4));
  // Blockers thin out progressively — more stages = less help
  const blockers = Math.min(
    size * size - 1,
    base.randomBlockers + Math.floor((stageNumber - 1) * 1.5)
  );
  // ...
}
```

Future Improvements

Gameplay

- **Undo move** — let the player take back the last placed blocker once per level, providing a safety valve against mis-clicks on high-pressure stages
- **Multiple dots** — spawn 2 or 3 dots simultaneously on higher levels, requiring the player to manage multiple escape vectors at once
- **Moving blockers** — introduce a cell type that slides one step per turn, forcing players to account for a dynamic, shifting board
- **Power-ups** — collectible cells that pause the dot for 2 turns, reveal its next planned move, or let you place 2 blockers simultaneously
- **Bomb tiles** — explosive cells that destroy adjacent blockers when the dot passes over them, punishing passive defense strategies

Persistence & Progression

- **`localStorage` high scores** — persist personal bests and Timed Mode records across browser sessions so progress survives page refresh
- **Level unlock system** — lock levels 6–10 behind completing the prior level, giving the difficulty curve narrative stakes
- **Run history** — store the last 5 Timed Mode runs with stage count and timestamps, displayed as a leaderboard on the start screen

Visuals & Polish

- **Dot path preview** — on hover over an empty cell, highlight in red the path the dot would take if *not* blocked there, letting players visualize consequences before committing
- **Win/lose particle effects** — a neon burst explosion on a successful trap; a screen shake + flash on escape — rather than text alone
- **Sound design** — Web Audio API for a thud on block placement, a ping on dot movement, a win jingle, and an escape alarm — all synthesized in JavaScript with no audio files needed
- **Theme switcher** — alternate palettes (deep ocean blue, matrix green, desert amber) toggled from the start screen using CSS custom property swaps
- **Animated dot personality** — subtle oscillation/wobble on the dot as it approaches the edge, communicating urgency without UI text

Technical

- **`localStorage` persistence** — save `state.bestByKey` to localStorage so personal bests survive page refresh
- **Service Worker + PWA** — cache all three files and the Google Font for full offline play; add a Web App Manifest so it can be installed as a home screen app on mobile
- **Keyboard accessibility** — arrow keys to move a cursor across the hex grid, `Space` to place a blocker, `Tab` to cycle UI buttons — with visible focus rings on every interactive element
- **AI hint system** — a "Show Hint" button that runs a one-step look-ahead to suggest the optimal single blocker placement, highlighting the recommended cell on the board
- **Replay sharing** — record each move as `{ row, col, timestamp }` and serialize to a URL hash parameter so players can share exact game replays as a link

- **`ResizeObserver` for canvas** — replace the `window.resize` event listener with the more precise `ResizeObserver` API targeting the canvas container

---

Author

danish1323

GITHUB DANISH1323

Built with zero dependencies, maximum neon, and deep respect for the classics.

If this project helped you, sparked ideas, or you just want to say hey — drop a on the repo. It means a lot.

INSERT COIN TO CONTINUE